

Projekt końcowy „Klastry obliczeniowe i rozwiązania chmurowe”

Zadanie 1 Walidacja integralności danych

```

from pyspark.sql.functions import col, regexp_extract, length

fastq_validation = fastq_clean.withColumn(
    "declared_length",
    regexp_extract(col("header"), r"length=(\d+)", 1).cast("int")
).withColumn(
    "actual_length",
    length(col("sequence"))
)

invalid_reads = fastq_validation.filter(
    col("declared_length") != col("actual_length")
)

print(f"Liczba niezgodnych rekordów: {invalid_reads.count()}")

invalid_reads.select(
    "record_id",
    "read_id",
    "declared_length",
    "actual_length"
).show(10, truncate=False)

```

▼ (4) Spark Jobs

- ▶ Job 15 [View](#) (Stages: 1/1)
- ▶ Job 16 [View](#) (Stages: 1/1, 1 skipped)
- ▶ Job 17 [View](#) (Stages: 1/1)
- ▶ Job 18 [View](#) (Stages: 1/1, 1 skipped)

▼ fastq_validation: pyspark.sql.dataframe.DataFrame

```

record_id: integer
header: string
quality: string
separator: string
sequence: string
read_id: string
declared_length: integer
actual_length: integer

```

▼ invalid_reads: pyspark.sql.dataframe.DataFrame

```

record_id: integer
header: string
quality: string
separator: string
sequence: string
read_id: string
declared_length: integer
actual_length: integer

```

Liczba niezgodnych rekordów: 0

record_id	read_id	declared_length	actual_length
-----------	---------	-----------------	---------------

- Ile Jobs zostało utworzonych? Czy widzimy zależność między liczbą wywołań akcji a liczbą zadań?

Zostały wykonane 4 Job (od 15 do 18). Liczba Jobów odpowiada liczbie wykonanych akcji (count() i show()), ponieważ każda akcja uruchamia wykonanie planu obliczeń Sparka. W Jobach 15 i 17 liczba tasków odpowiada liczbie domyślnych partycji DataFrame (7). W Jobach 16 i 18 wykonany został tylko jeden task, ponieważ były to etapy kończące wykonanie akcji. Wszystkie taski miały poziom lokalności PROCESS_LOCAL, co oznacza, że dane były przetwarzane lokalnie, bez konieczności przesyłania ich między węzłami.

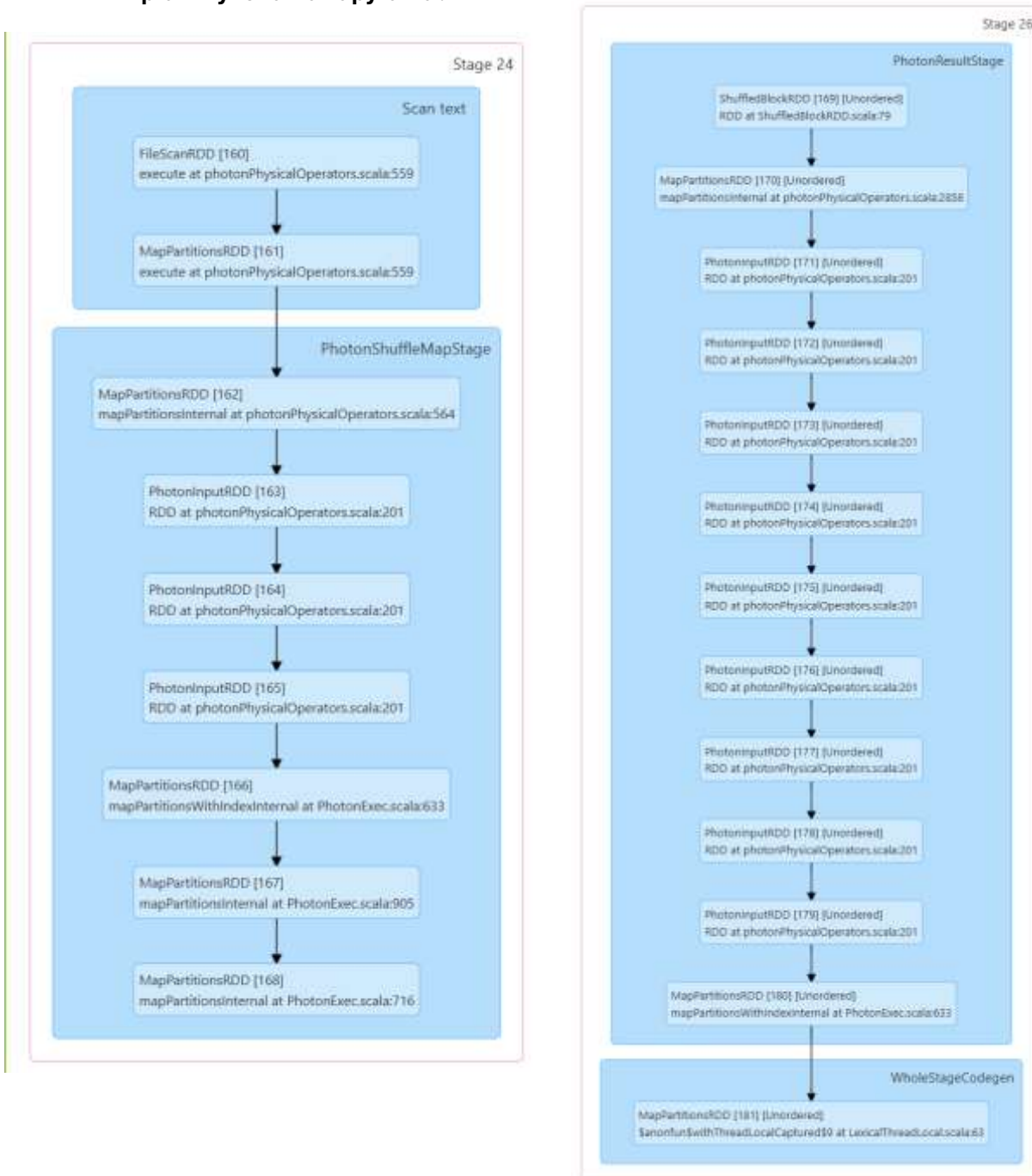
- Które operacje były transformacjami, a które akcjami? Wypiszmy je, aby utrwalić tę kluczową różnicę.

Transformacje to: withColumn(), filter(), select()

Akcje: count(), show()

Transformacje tworzą plan wykonania a akcje uruchamiają jego wykonanie.

- Sprawdźmy DAG Visualization dla tego Joba. Ile etapów (RDD/DataFrame) tworzy plan wykonania zapytania?



FileScanRDD, MapPartitionsRDD, PhotonInputRDD, ShuffledBlockRDD

- Jedyny Stage wykonał 7 tasków, co odpowiada domyślnej liczbie 7 partycji DataFrame. Oznacza to, że Spark tworzy jeden task dla każdej partycji danych. Dzięki temu partycje mogą być przetwarzane równolegle, co zwiększa wydajność obliczeń.

[illegible]

- Zadanie 2 było szybsze od Zadania 1, ponieważ wykorzystuje prostą operację `contains("#")`, która tylko sprawdza obecność jednego znaku w ciągu. Natomiast w Zadaniu 1 wykonywane były bardziej złożone operacje `regex_extract()` oraz `length()`, które wymagają większej liczby obliczeń, dlatego ich wykonanie trwa dłużej.

- **Czy Spark wykorzystał cache? Przyjrzyjmy się liście stage'ów. Czy któryś z nich ma przy sobie zieloną etykietę "skipped"? Jeśli tak, oznacza to, że Spark nie musiał ponownie czytać danych z dysku, ponieważ skorzystał z wyniku zapisanego w pamięci przez .cache() w poprzednim kroku.**

Tak, spark wykorzystał cache. W liście Jobów i Stage'ów widać zieloną etykietę „skipped”, co oznacza, że część etapów została pominięta. Spark nie musiał ponownie odczytywać danych z dysku, ponieważ skorzystał z danych zapisanych wcześniej w pamięci dzięki użyciu .cache().

- **Jaki był Locality Level tasków? W szczegółach Stage'a znajdziemy informację o "Locality Level". Czy dominowało PROCESS_LOCAL (najlepsza opcja, dane w pamięci tego samego procesora)? Jeśli tak, dlaczego?**

We wszystkich taskach dominował PROCESS_LOCAL. Oznacza to, że dane były dostępne lokalnie w pamięci tego samego procesu, dzięki czemu Spark nie musiał odczytywać ich z dysku ani przysyłać przez sieć. Stało się tak, ponieważ dane zostały zapisane w pamięci za pomocą .cache().

- **Ile rekordów (Records) przetworzył każdy Task? Czy liczba ta jest zgodna z Twoimi oczekiwaniami dotyczącymi podziału danych na partycje?**

Jobs (7)

Job ID	Task ID	Attempt	Event	Locality Level	Duration (s)	Host	Task	Locality Group	Duration (ms)	Result	GC Time	Shuffle Read Size (MB)	Shuffle Write Size (MB)	Input Size / Records	Shuffle Write Size / Records
0	0	0	JobStart	PROCESS_LOCAL	0.000	10.10.10.1	JobStart	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000
0	1	0	TaskStart	PROCESS_LOCAL	0.000	10.10.10.1	TaskStart	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000
0	2	0	TaskStart	PROCESS_LOCAL	0.000	10.10.10.1	TaskStart	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000
0	3	0	TaskStart	PROCESS_LOCAL	0.000	10.10.10.1	TaskStart	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000
0	4	0	TaskStart	PROCESS_LOCAL	0.000	10.10.10.1	TaskStart	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000
0	5	0	TaskStart	PROCESS_LOCAL	0.000	10.10.10.1	TaskStart	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000
0	6	0	TaskStart	PROCESS_LOCAL	0.000	10.10.10.1	TaskStart	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000

Shuffle Remote Reads	Input Size / Records	Shuffle Write Size / Records
0.0 B	126.1 MB / 1417497	48.5 MB / 1417497
0.0 B	126.1 MB / 1412819	48.4 MB / 1412819
0.0 B	126.1 MB / 1410285	48.4 MB / 1410285
0.0 B	126.1 MB / 1398101	48.2 MB / 1398101
0.0 B	126.1 MB / 1398102	48.2 MB / 1398102
0.0 B	126.1 MB / 1398101	48.3 MB / 1398101
0.0 B	77.2 MB / 843599	28.7 MB / 843599

U mnie jest łącznie 9 278 504 rekordy i 7 tasków, czyli ok. 1,33 mln rekordów na task. (ostatni ma jedynie 843 599)

Zadanie 3

```
from pyspark.sql.functions import col, length

quality_analysis = (
    fastq_clean
    .withColumn("has_low_quality", col("quality").contains("N"))
    .withColumn("seq_length", length(col("sequence")))
    .groupBy("has_low_quality", "seq_length")
    .count()
    .orderBy("has_low_quality", "seq_length")
    .cache()
)

print(f"Liczba grup: {quality_analysis.count()}")

print(f"Liczba różnych długości sekwencji: {quality_analysis.select('seq_length').distinct().count()}")

quality_analysis.show(20, truncate=False)
```

▼ (5) Spark Jobs

- ▶ Job 23 [View](#) (Stages: 1/1)
- ▶ Job 24 [View](#) (Stages: 1/1, 1 skipped)
- ▶ Job 25 [View](#) (Stages: 1/1, 1 skipped)
- ▶ Job 26 [View](#) (Stages: 1/1, 1 skipped)
- ▶ Job 27 [View](#) (Stages: 1/1, 1 skipped)

▶ quality_analysis: pyspark.sql.dataframe.DataFrame = [has_low_quality: boolean, seq_length: integer ... 1 more field]

Liczba grup: 2
Liczba różnych długości sekwencji: 1

has_low_quality	seq_length	count
False	151	2313942
true	151	5684

- **Ile Jobs zostało utworzonych? Wypiszmy każdą akcję z kodu i przypisz jej numer Joba z UI.**

Zostało utworzonych 5 Jobs od 23 do 27.

Akcja z kodu	Nr. job
Quality_analasis.count()	23
Quality_analasis.select("seq_length").distinct().count()	24
Quality_analasis.show(20, truncate=False)	25
Cache()	26
(show)	27

- **Które Jobs wykorzystały cache? Zidentyfikujmy Joby, których etapy zostały pominięte ("skipped"). Dlaczego tak się stało?**

Joby, które wykorzystały cache to: 24,25,26,27

Po wykonaniu pierwszej akcji (`quality_analysis.count()`, Job 23) DataFrame został zapisany w pamięci dzięki wywołaniu `.cache()`. Kolejne akcje (`distinct().count()` i `show()`) nie musiały ponownie odczytywać danych z dysku. Spark wykorzystał dane znajdujące się w pamięci, dlatego część etapów została oznaczona jako `skipped`, co skróciło czas wykonania kolejnych Jobów.

- **Efektywność cache:** porównajmy czas wykonania pierwszego Joba (który musiał obliczyć wszystko od zera) z czasami kolejnych Jobów (które czytały z cache). Jaki jest zysk?

Pierwszy Job (23) trwał 11 sekund a każdy kolejny 3 sekundy oznacza to, że proces był około 3 razy szybszy. Pierwszy Job musiał wykonać wszystkie transformacje i zapisać wynik do pamięci, dlatego trwał najdłużej. Kolejne Joby korzystały z danych zapisanych w cache.

- **Locality Level:** Czy wszystkie taski w Jobach korzystających z cache miały **PROCESS_LOCAL**? Jeśli nie, spróbujmy określić, dlaczego niektóre taski musiały czytać dane z pamięci na innym węźle (**NODE_LOCAL**)

Wszystkie taski w Jobach wykorzystują **PROCESS_LOCAL**

- **Porównajmy Input Size między różnymi Jobami. Dlaczego Jobs korzystające z cache mają bardzo mały lub zerowy rozmiar wejściowy?**

Joby korzystające z cache mają bardzo mały lub zerowy Input Size, ponieważ Spark nie odczytuje ponownie danych z pliku. Korzysta z danych zapisanych wcześniej w pamięci (cache)

- **Narysujmy schemat Stage'ów dla pierwszego Joba (tego z groupBy):**

Stage 1: Jakie operacje się tu odbywały? Czy widzimy w metrykach Shuffle Write? (to etap, na którym taski przygotowują dane do agregacji).

Stage 2: Jakie operacje? Czy widzimy Shuffle Read? (to etap agregacji, na którym taski czytają dane przygotowane przez Stage 1).

- skończył mi się darmowy kredyt, więc nie jestem w stanie zamieścić zdjęcia